Object Oriented Programming for Data Processing
3 July 2026

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 13:15 as a single `zip` file.

C++ evaluation will be based on: correct syntax, proper return types and arguments of functions, data members and class interfaces, setters/getters, avoiding unnecessary void functions, correct mathematical expressions and physical units, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, correct mathematical expressions and physical units, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general, favouring modular coding over sequential coding.

Part 1 – C++

Implement a class `DualNumber` to represent and manipulate *dual numbers*, expressions of the form $a + b\epsilon$, where $a, b \in \mathbb{R}$ and ϵ is a nilpotent element such that $\epsilon \neq 0$ and $\epsilon^2 = 0$.

The real number a is called the real part, and b is called the dual part. Dual numbers constitute a 2-dimensional vector space over $\mathbb{R}$ with $\{1, \epsilon\}$ as a basis and the following mathematical properties for addition and multiplication:

$$(a_1 + b_1\epsilon) + (a_2 + b_2\epsilon) = (a_1 + a_2) + (b_1 + b_2)\epsilon \tag{1}$$
$$\lambda(a + b\epsilon) = (\lambda a) + (\lambda b)\epsilon \tag{2}$$
$$(a_1 + b_1\epsilon)(a_2 + b_2\epsilon) = a_1a_2 + (a_1b_2 + a_2b_1)\epsilon\,. \tag{3}$$

Implement the following methods in `DualNumber`.

- A regular constructor (with default values for the real and dual parts) and a copy constructor.
- Appropriate getters and setters.
- Overload the `<<` operator to print the dual number in the format `a + b*eps`.
- Overload the `+` and `*` operators appropriately to operate between two `DualNumber` instances, and between a real number and a `DualNumber` instance (in both orderings).
- A conjugation method that returns $a - b\epsilon$.

Your submitted material must include a file `app.cpp` that showcases each method you implemented.

Part 2 – Python

In quantum mechanics, alpha particles escape unstable heavy nuclei via barrier penetration. Treating the nucleus as a spherical potential boundary, this rate of particle loss can be modeled via the Gamow factor expression:

$$\frac{dN}{dt} = -\nu \exp\left(-\frac{\pi Z e^2}{2\pi\epsilon_0 \hbar v}\right) N = -\lambda N\,, \tag{4}$$

where $N = N(t)$ represents remaining active parent nuclei, ν ($\sim 10^{-21}$–$10^{-22}\,\mathrm{Hz}$) is the internal collision frequency, Z is the atomic number, and v is the final particle escape velocity ($\sim$5%–10% the speed of light). The physical constants are $\epsilon_0 = 8.854 \times 10^{-12}\,\mathrm{F/m}$, $\hbar = 1.054 \times 10^{-34}\,\mathrm{J{\cdot}s}$, and $e = 1.602 \times 10^{-19}\,\mathrm{C}$.

Use a Python script or Jupyter Notebook to perform the following tasks.

1. Modelling and simulating decays

   - Write a routine to evolve this decay system in time and to evaluate the radioactive half-life $T_{1/2}$ by integrating until the initial count $N(0)$ drops by one half.
   - Implement a visualization function showcasing the remaining fraction of active isotopes vs time, and the log-scale plot of the Total Nuclear Activity ($\mathcal{A} = -dN/dt$).
   - Evaluate and present comparative lifecycle decay profiles for:
     (a) a Superheavy Nucleus Type-A ($Z = 118$, high internal velocity v)
     (b) an Intermediate Actinide ($Z = 92$, moderate internal velocity)
     (c) a Light Unstable Isotope ($Z = 20$, lower internal velocity scale).

2. Bayesian Inference on nuclear activity data

   - Consider the dataset `activity_data.txt` containing observed nuclear activity ($\mathcal{A}_i$) at discrete time intervals (t_i) with associated experimental measurement errors (σ_i). Write functions to read in and visualize this raw data.
   - Infer $\boldsymbol{\theta} = (\log_{10}\nu, v)$ from the data with the `emcee` package by implementing reasonable uniform priors for $\log_{10}\nu$ and v, and the log-likelihood function

     $$\ln \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{2}\sum_{i=1}^{N_{\mathrm{data}}} \left\{ \frac{[\mathcal{A}_i - \mathcal{A}_{\mathrm{th}}(t_i, \boldsymbol{\theta})]^2}{\sigma_i^2} + \ln(2\pi\sigma_i^2) \right\},$$

     where $\mathcal{A}_{\mathrm{th}}(t_i, \boldsymbol{\theta}) = \lambda N(0) \exp(-\lambda t_i)$ and λ depends on $\boldsymbol{\theta}$ as specified by Eq. (4). Assume $N(0) = 1.0 \times 10^{23}$ and $Z = 92$.
   - Plot your results with the `corner` package; extract the mean and 90% credible intervals for $\log_{10}\nu$ and v.